

EF Core is too slow

How to actually make it fast



01

Add Database Indexes

- EF Core doesn't automatically create indexes.
- Add them manually for columns used in WHERE, JOIN, and ORDER BY.

```
public sealed class AppDbContext : DbContext
{
    public DbSet<Order> Orders => Set<Order>();

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Order>(b =>
        {
            b.HasKey(x => x.Id);

            b.HasIndex(x => new
            {
                x.TenantId,
                x.Status,
                x.CreatedAtUtc
            })
                .HasDatabaseName("IX_Orders_Tenant_Status_CreatedAt");

            b.Property(x => x.TotalAmount).HasPrecision(18, 2);
        });
    }
}
```



Optimize Query Projections

- Stop fetching full entities when you only need a few fields.
- Use `.Select()` to return only what's required.
- Smaller payloads = faster materialization and less memory usage.

```
public sealed class OrdersController(AppDbContext _db)
{
    public async Task<IReadOnlyList<OrderSummaryDto>> GetAsync(
        CancellationToken ct)
    {
        return await _db.Orders
            .AsNoTracking()
            .OrderByDescending(x => x.CreatedAtUtc)
            .Select(x => new OrderSummaryDto(
                x.Id,
                x.Customer.Name,
                x.Items.Count))
            .Take(100)
            .ToListAsync(ct);
    }
}
```



03

Use AsNoTracking For Read-Only Queries

- Tracked entities consume memory and CPU.
- Add `.AsNoTracking()` for read-only queries.

```
public sealed class ProductsQuery(AppDbContext _db)
{
    public async Task<IReadOnlyList<ProductDto>> SearchAsync(
        string name, CancellationToken ct)
    {
        return await _db.Products
            .AsNoTracking()
            .Where(x => x.Name.Contains(name))
            .OrderBy(x => x.Name)
            .Select(x => new ProductDto(x.Id, x.Name))
            .Take(50)
            .ToListAsync(ct);
    }
}
```



List Number Of Navigations

- Too many .Include() calls can cause N+1 queries or massive joins.
- Include only what's necessary.
- Filter includes where possible

```
public sealed class OrdersQuery(AppDbContext db)
{
    public async Task<IReadOnlyList<Order>> GetRecentAsync(
        Guid tenantId, CancellationToken ct)
    {
        return await db.Orders
            .AsNoTracking()
            .Where(x => x.TenantId == tenantId)
            .OrderByDescending(x => x.CreatedAtUtc)
            .Select(x => new
            {
                x.Id,
                x.CreatedAtUtc,
                CustomerName = x.Customer.Name,
                Items = x.Items
                    .Where(i => i.IsActive)
                    .Select(i => new
                    {
                        i.ProductId,
                        i.Quantity
                    })
                    .ToList()
            })
            .Take(50)
            .ToListAsync(ct);
    }
}
```



05

Avoid Large IN / Contains Filters

- Fetch IDs in smaller batches.
- Huge Contains() lists generate slow SQL IN clauses.
- Or store filter data in a temporary table and join it instead.

```
public sealed class OrdersBatchQuery(AppDbContext db)
{
    private const int BatchSize = 500;

    public async Task<IReadOnlyList<Order>> GetAsync(
        IReadOnlyList<Guid> orderIds,
        CancellationToken ct)
    {
        var result = new List<Order>(orderIds.Count);

        foreach (var batch in orderIds.Chunk(BatchSize))
        {
            var items = await db.Orders
                .AsNoTracking()
                .Where(x => batch.Contains(x.Id))
                .Select(...)
                .ToListAsync(ct);

            result.AddRange(items);
        }

        return result;
    }
}
```



Use Pagination For Large Datasets

- Don't load thousands of records into memory.
- Use `.Skip()` and `.Take()` for offset-based pagination.
- Or switch to cursor-based pagination — it's faster and index-friendly.

```
public sealed class OrdersPageQuery(AppDbContext db)
{
    public async Task<IReadOnlyList<Order>> GetAsync(
        int page,
        int pageSize,
        CancellationToken ct)
    {
        return await db.Orders
            .AsNoTracking()
            .OrderByDescending(x => x.CreatedAtUtc)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .ToListAsync(ct);
    }
}
```



07

Use Compiled Queries

- Cache LINQ-to-SQL translation with EF `.CompileQuery()`.
- Frequent queries can be precompiled.
- It's perfect for queries used in loops or called often.

```
public static class CompiledOrderQueries
{
    public static readonly Func<AppDbContext, Guid, CancellationToken, Task<List<Order>>> GetByTenantAsync =
        EF.CompileAsyncQuery(
            (AppDbContext db, Guid tenantId, CancellationToken ct) =>
                db.Orders
                    .AsNoTracking()
                    .Where(x => x.TenantId == tenantId)
                    .OrderByDescending(x => x.CreatedAtUtc)
                    .Take(50)
                    .ToList());
}
```

```
public sealed class OrdersQuery(AppDbContext db)
{
    public Task<List<Order>> GetRecentAsync(Guid tenantId, CancellationToken ct)
    {
        return CompiledOrderQueries.GetByTenantAsync(db, tenantId, ct);
    }
}
```



Use SplitQuery to Avoid Cartesian Explosion

- Multiple `.Include()` joins can cause data duplication.
- Add `.AsSplitQuery()` to break large joins into smaller queries.

```
public sealed class OrderDetailsQuery(AppDbContext db)
{
    public async Task<Order> GetAsync(Guid orderId, CancellationToken ct)
    {
        return await db.Orders
            .AsNoTracking()
            .AsSplitQuery()
            .Include(x => x.Customer)
            .Include(x => x.Items)
            .ThenInclude(x => x.Product)
            .FirstAsync(x => x.Id == orderId, ct);
    }
}
```



09

Introduce Caching

- Don't hit the database for frequently accessed data.
- For .NET 9+, try HybridCache — it solves cache stampede issues.

```
public sealed class ProductCacheQuery(
    AppDbContext db,
    HybridCache cache)
{
    public async Task<Product?> GetAsync(
        Guid productId, CancellationToken ct)
    {
        return await cache.GetOrCreateAsync(
            $"product:{productId}",
            async token =>
            {
                return await db.Products
                    .AsNoTracking()
                    .FirstOrDefaultAsync(x => x.Id == productId, token);
            },
            ct);
    }
}
```



Use Raw SQL For Complex Operations

- Sometimes SQL just wins.
- You get full control over the query plan.

```
public sealed class RevenueQuery(AppDbContext db)
{
    public async Task<IReadOnlyList<MonthlyRevenue>> GetAsync(
        DateTime fromUtc,
        DateTime toUtc,
        CancellationToken ct)
    {
        return await db.Database
            .SqlQuery<MonthlyRevenue>(
                """
                SELECT date_trunc('month', created_at_utc) AS month,
                       SUM(total_amount) AS revenue
                FROM orders
                WHERE created_at_utc BETWEEN {0} AND {1}
                GROUP BY 1
                """,
                fromUtc,
                toUtc)
            .ToListAsync(ct);
    }
}
```



Find this useful?



Repost if you found this insightful



Save for future inspiration



Subscribe to my newsletter, [AntonDevTips](#),
for more insights on .NET and Architecture